

モニュメント (Monuments)

サマルカンドにあるレジスタン広場には、広場の中心を通る直線の上に N 体のモニュメントが位置している。モニュメントには 0 から $N - 1$ までの番号が付けられている。モニュメント i ($0 \leq i < N$) は最初整数の座標 $X[i]$ に位置している。広場の中心は座標 0 にある。

建築家は広場の中心に関する完全な対称性を要求している。建築家は何体かのモニュメント (0 体であることも許される) を動かし、最終的な配置が座標 0 に関して対称になるようにしたいと考えている。より厳密には、

- すべてのモニュメントは整数の座標に置かれなければならない。
- それぞれの整数座標には **0 体, 1 体あるいは複数体** のモニュメントを配置することができる。
- すべての整数 $x > 0$ について、座標 x に置かれているモニュメントの数は座標 $-x$ に置かれているモニュメントの数に等しくなければならない。座標 0 に置かれているモニュメントの数はいくつでも良い (特に 0 体でも良い) 。

しかしながら、 M 体のモニュメントは太古のものであり、動かすには脆すぎる。これらのモニュメントの番号は $P[0], P[1], \dots, P[M - 1]$ である。これら M 体のモニュメントは最初の位置から動かすことはできない。残りの $N - M$ 体のモニュメントは任意の整数座標に動かすことができる。モニュメントは移動の最中でお互いに全く干渉しない。

ある 1 つのモニュメントを座標 a から座標 b へ動かすのにかかるコストは絶対値 $|a - b|$ に等しい。全体のコストは、動かされたモニュメントそれぞれのコストの総和である。

あなたの課題は、モニュメントの位置を座標 0 に関して対称にするために必要なコストの総和の最小値を求めるか、あるいは与えられた条件のもとではそのような対称な配置を実現できないことを判定することである。

実装の詳細

あなたは以下の関数を実装する必要がある。

```
long long get_cost(std::vector<int> X, std::vector<int> P)
```

- X : 長さ N の広義単調増加な配列であり、モニュメントの最初の座標を表す。
- P : 長さ M の狭義単調増加な配列であり、太古のモニュメントの番号を表す。
- この関数は、それぞれのテストケースにおいてちょうど 1 回呼び出される。

この関数は整数を 1 つ返す必要がある。配置を対称にすることが不可能であるときは -1 を、可能であるときは必要な最小のコストの総和を返す必要がある。

制約

- $1 \leq N \leq 500\,000$
- $0 \leq M \leq N$
- $-10^9 \leq X[0] \leq X[1] \leq \dots \leq X[N-1] \leq 10^9$
- $0 \leq P[0] < P[1] < \dots < P[M-1] < N$

小課題

小課題	得点	追加の制約
1	3	$M = N$
2	4	$M = 0$
3	5	$0 \leq j < M$ であるようなそれぞれの j について $X[P[j]] < 0$.
4	6	$N \leq 10$
5	5	$N \leq 19$
6	5	$N \leq 32$
7	13	$N \leq 200$
8	17	$N \leq 4000$
9	13	$M \leq 4000$
10	29	追加の制約はない.

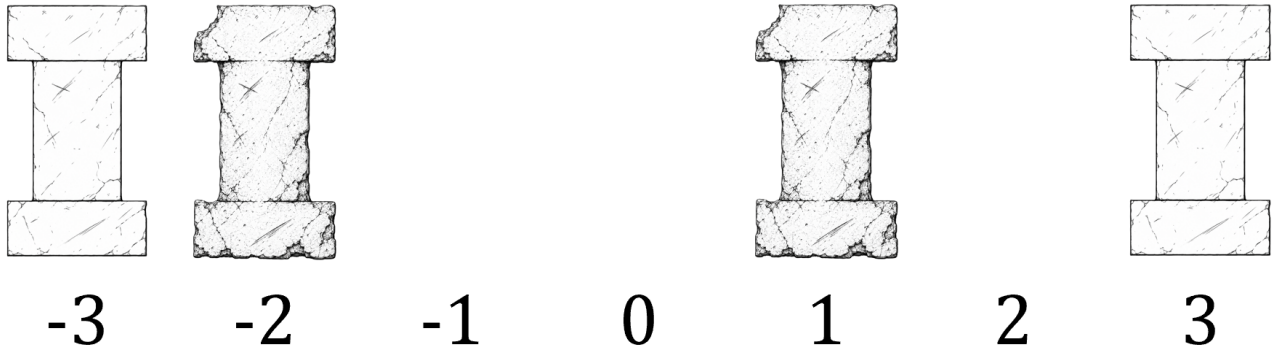
例

例 1

次の呼び出しを考える。

```
get_cost([-3, -2, 1, 3], [1, 2])
```

最初のモニュメントの配置は次の図で表される。

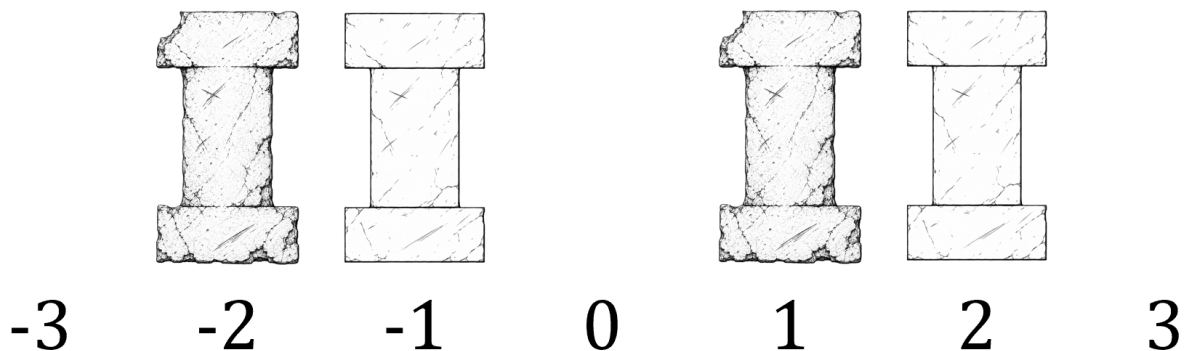


$N = 4$ つのモニュメントがあり、最初は座標 $[-3, -2, 1, 3]$ に置かれている．太古のモニュメントの番号は $P[0] = 1$ と $P[1] = 2$ である．つまり、座標 $X[P[0]] = -2$ と $X[P[1]] = 1$ に置かれているモニュメントは動かすことができない．座標 -3 と 3 に置かれている残りのモニュメントは動かすことができる．

最小のコストで配置を対称にするためには、次のようにモニュメントを動かすことになる．

- モニュメントを座標 -3 から -1 へコスト $|(-3) - (-1)| = 2$ で動かす．
- モニュメントを座標 3 から 2 へコスト $|3 - 2| = 1$ で動かす．

これらの移動の後、配置は対称になる．



移動のコストの合計は $2 + 1 = 3$ であるため、この関数は 3 を返さなければならない．

例 2

次の呼び出しを考える．

```
get_cost([2, 2, 2, 3], [])
```

ここでは $M = 0$ であり、これは太古のモニュメントは存在せずすべてのモニュメントを動かすことができることを意味する．最小のコストを達成する 1 つの方法は、すべてのモニュメントを座標 0 に移動させることである．それぞれの移動のコストは、

- モニュメント $0, 1, 2$ に関しては $|2 - 0| = 2$ である．
- モニュメント 3 に関しては $|3 - 0| = 3$ である．

コストの総和は $2 + 2 + 2 + 3 = 9$ である。したがって、この関数は 9 を返さなければならない。同じコストを達成する別の動かし方も存在することに注意せよ。

例 3

次の呼び出しを考える。

```
get_cost([1, 2, 3, 4], [0, 1, 2, 3])
```

4 つすべてのモニュメントが太古のものであり、動かすことができない。モニュメントの座標はすべて正であるため、配置を座標 0 に関して対称にすることは不可能である。したがって、この関数は -1 を返さなければならない。

採点プログラムのサンプル

入力形式：

```
N M
X[0] X[1] ... X[N-1]
P[0] P[1] ... P[M-1]
```

$M = 0$ であるときに 3 行目が空になり得ることに注意せよ。

出力形式：

```
C
```

C は `get_cost` によって返された値を表す。